

CS234 REINFORCEMENT LEARNING DEEP READING BRIEF

Source course: Stanford CS234, Winter 2026 (<https://web.stanford.edu/class/cs234/> and <https://web.stanford.edu/class/cs234/modules.html>).

I could not run the requested local deepbrief-daily skill because `/Users/liuzikai/Documents/GitHub/deepbrief/.agents/skills/deepbrief-daily/SKILL.md` is not present in this container. This file is the fallback deep-brief artifact: it incorporates the course home page, lecture-materials page, lecture slide PDFs, additional readings, discussion/check-your-understanding prompts, guest materials, and reference links into one study document. The generated target PDF is `notes/cs234-reading-brief.pdf`.

REPRODUCIBLE ARTIFACTS

- Target PDF: `notes/cs234-reading-brief.pdf`.
- Source brief: `notes/cs234-reading-brief.md`.
- Full PDF bundle downloader: `scripts/download_cs234_pdfs.py`.
- PDF generator: `scripts/generate_cs234_summary_pdf.py`.

Run:

```
bash
python3 scripts/download_cs234_pdfs.py --out-dir downloads/cs234-pdfs
python3 scripts/generate_cs234_summary_pdf.py
```

The downloader writes a JSON manifest with per-file status. In this container the course hosts are blocked by the outbound proxy, but the manifest is explicit and ready for a network that can reach Stanford, `cs229.stanford.edu`, `incompleteideas.net`, and `tor-lattimore.com`.

COMPLETE COURSE-MATERIAL MANIFEST

Topic | Official materials to download/read | Additional readings/references | What to extract

Introduction to RL | `lecture1pre.pdf`, `lecture1post.pdf` | Sutton & Barto Ch. 1; CS229 linear algebra review; CS229 probability review; CS231n Python/Numpy tutorial | RL as experience-driven sequential decision-making under uncertainty; optimization, delayed consequences, exploration, and generalization; how to formulate state/action/reward/dynamics for applications.

Tabular MDP planning | `lecture2pre.pdf`, `lecture2post.pdf` | Sutton & Barto Ch. 3, 4.1-4.4 | Markov reward/decision processes, policies, returns, value functions, Bellman expectation/optimality equations, policy evaluation, policy improvement, policy iteration, value iteration, and contraction/fixed-point intuition.

Tabular RL policy evaluation | `lecture3pre.pdf`, `lecture3post.pdf` | Sutton & Barto 5.1, 5.5, 6.1-6.3 | Estimating a policy's value from direct experience; Monte Carlo first/every visit evaluation; temporal-difference learning; bootstrapping; certainty-equivalence/batch ideas; bias-variance tradeoffs.

Q-learning / model-free control | `lecture4pre.pdf`, `lecture4post.pdf` | Sutton & Barto 5.2, 5.4, 6.4-6.5, 6.7 | SARSA versus Q-learning, on-policy versus off-policy control, epsilon-greedy exploration, GLIE intuition, TD control targets, and why function approximation changes stability guarantees.

Policy gradient I | `lecture5pre.pdf`, `lecture5post.pdf` | Sutton & Barto Ch. 13 | Directly optimizing stochastic policies; policy-gradient theorem; likelihood-ratio/score-function gradients; REINFORCE; baselines; return-to-go; variance reduction.

Policy gradient II | [lecture6pre.pdf](#), [lecture6post.pdf](#) | Sutton & Barto Ch. 13; Joshua Achiam-derived advanced PG material | Advantage functions, actor-critic structure, natural/trust-region style motivation, practical instability, and why critic quality controls actor update quality.

Policy gradients and imitation | [lecture7pre.pdf](#), [lecture7post.pdf](#) | Sutton & Barto Ch. 13 plus imitation-learning references | Policy search continuation, supervised imitation, distribution shift, behavior cloning limits, and transition into DAgger/reward-learning views.

Imitation, RLHF, DPO, batch/offline RL | [lecture8pre.pdf](#), [lecture8post.pdf](#) | Ross et al. DAgger; preference-learning/RLHF references; Bradley-Terry preference model | Compounding errors, dataset aggregation, reward ambiguity, pairwise preference learning, RL from human feedback, direct preference optimization, and offline-policy risk.

Data-efficient RL I | [lecture9pre.pdf](#), [lecture9post.pdf](#) | Bandit Algorithms Section 7.1 | Regret/sample-efficiency framing, optimism under uncertainty, confidence intervals, PAC/sample-complexity thinking, and why random exploration is often inefficient.

Fast RL / data-efficient RL II / ethics | [lecture10post.pdf](#), [ethics_society_234_2.pdf](#) | Bandit Algorithms Section 7.1; ethics and society guest material | Strategic exploration, posterior/optimistic methods, deployment incentives, societal feedback loops, reward misspecification, and responsible evaluation.

Fast RL / strategic exploration | [lecture11pre.pdf](#), [lecture11post.pdf](#) | Bandit Algorithms Section 7.1 | Information-directed action choice, uncertainty-aware learning, optimism/posterior sampling connections, and how exploration methods scale imperfectly in deep settings.

Fast RL continued | [lecture12pre.pdf](#), [lecture12post.pdf](#) | Bandit Algorithms Section 7.1 | Continued efficient RL: exploration bonuses, confidence sets, model uncertainty, intrinsic motivation, deep approximations, and empirical/theoretical tradeoffs.

Monte Carlo Tree Search | [lecture13pre.pdf](#), [lecture13post.pdf](#) | AlphaGo/AlphaZero papers referenced in slides | Simulation-based search, simple Monte Carlo search, expectimax, UCT, search-tree backups, self-play, value/policy networks, and the role of local computation.

MCTS continuation / guest ethics part 2 | [lecture14pre.pdf](#), [lecture14post.pdf](#), [ethics_society_234_2.pdf](#) | AlphaGo/AlphaZero and ethics references | Planning-learning integration, self-play curricula, neural MCTS variants, and ethical evaluation beyond reward maximization.

RL guest lecture | [ShaneGuCS234_2026.pdf](#) | World-modeling/model-based RL literature | Learned dynamics/world models, compact predictive representations, planning in latent space, model error, and why model-based RL can be sample-efficient but fragile.

DEEP SYNTHESIS BY COURSE ARC

1. WHAT RL IS AND WHEN TO USE IT

The course begins by separating reinforcement learning from standard non-interactive machine learning. RL is about choosing actions that affect future observations, rewards, and opportunities. The central object is an agent-environment loop: state or observation arrives, the agent chooses an action, the environment transitions, and reward arrives. Four features appear repeatedly: optimization, delayed consequences, exploration, and generalization. The discussion prompts in lecture 1 ask students to model an AI tutor as a decision process; the point is that state, action, reward, and dynamics must be explicit before an RL algorithm is meaningful.

A practical rule: use RL when the problem involves sequential decisions, delayed consequences, and data that depends on the learner's choices. If expert-labeled independent examples are enough, supervised learning is simpler. If a known model is available and no learning is needed, planning may suffice. If there is a fixed logged dataset, offline RL or imitation may be safer than online trial-and-error.

2. MDPS, VALUE FUNCTIONS, AND PLANNING

The MDP section provides the mathematical foundation for the rest of the course. A finite discounted MDP consists of states, actions, transitions, rewards, and discount factor. A policy maps states or histories to actions. Returns sum discounted future rewards. State values and action values are expectations of return. Bellman equations make the recursive structure explicit: today's value is immediate reward plus discounted next value.

With a known model, dynamic programming can evaluate and improve policies. Policy iteration alternates evaluation and improvement. Value iteration directly applies Bellman optimality backups. The reason these methods work in the tabular discounted setting is contraction: repeated backups approach a unique fixed point. The limitation is scale; exact dynamic programming becomes impossible for large state/action spaces or unknown dynamics.

3. LEARNING POLICY VALUES FROM EXPERIENCE

Lecture 3 removes the known-model assumption and asks how to evaluate a policy from sampled trajectories. Monte Carlo policy evaluation averages observed returns after complete episodes. It is conceptually simple, does not require a Markov model, and can be first-visit or every-visit. Its weakness is high variance and the need for episode termination.

Temporal-difference methods update online using bootstrapped targets such as $r + \gamma V(s')$. TD mixes sampling and dynamic programming: it uses real transitions but substitutes the current value estimate for the rest of the future. This introduces bias but often lowers variance and works before an episode ends. The key discussion issue is not "MC versus TD is always better," but which estimator's bias/variance/online properties fit the environment.

4. CONTROL: SARSA, Q-LEARNING, AND EXPLORATION

Control asks how to improve behavior, not just evaluate a fixed policy. Action-value functions are central because they let the agent compare actions without a model. SARSA updates toward the action actually selected by the behavior policy and is on-policy. Q-learning updates toward the greedy next action and is off-policy. Both depend on sufficient exploration; greedy behavior can lock in bad estimates.

The lecture notes use this transition to introduce function approximation. Tabular convergence arguments assume each state-action pair can be represented and visited. Once values are approximated, especially with neural networks, generalization is helpful but instability appears. The "deadly triad" intuition is already visible: bootstrapping plus off-policy learning plus function approximation can diverge or overestimate.

5. POLICY GRADIENTS AND ACTOR-CRITIC METHODS

Policy-gradient lectures move from value-based control to direct policy optimization. A stochastic policy with parameters can be optimized using the policy-gradient theorem: actions whose sampled outcomes are better than expected should become more likely. REINFORCE is the core estimator; return-to-go and baselines reduce variance. A baseline does not change the expected gradient if it does not depend on the action.

Actor-critic methods add a learned value function. The actor changes the policy; the critic estimates value or advantage. Good critics reduce variance and improve sample efficiency, but

bad critics bias or destabilize updates. Trust-region, natural-gradient, and clipped-objective intuitions all respond to the same practical problem: a single policy-gradient step can make the policy worse, so updates should be controlled and evaluated.

6. IMITATION LEARNING, REWARD LEARNING, RLHF, AND OFFLINE RL

The imitation/RLHF part emphasizes that “learn from humans” is not one method. Behavior cloning treats demonstrations as supervised (state, action) examples, but its iid assumption breaks because learned policies visit states induced by their own errors. Errors compound over time. DAgger fixes this by aggregating expert labels on states visited by the learner.

Reward learning asks a different question: if we see expert behavior, what reward explains it? The answer is usually non-unique. Preference learning and RLHF collect comparisons or feedback and fit a reward/preference model. The Bradley-Terry style view makes pairwise comparisons probabilistic. DPO-like methods shortcut parts of reward-model-plus-RL pipelines. Offline RL adds another constraint: policies must not exploit unsupported actions in the dataset, because value estimates off the data distribution can be arbitrarily optimistic.

7. DATA-EFFICIENT RL AND EXPLORATION

The data-efficient RL lectures reframe learning quality in terms of regret, sample complexity, and useful information gathering. Random exploration is easy but often wasteful. Optimism under uncertainty acts as if uncertain actions may be good, encouraging the learner to test them. Posterior sampling samples a plausible model/value function and acts optimally under that sample. Count-based bonuses, confidence intervals, ensembles, and intrinsic rewards are different ways to operationalize uncertainty.

Bandit Algorithms Section 7.1 supplies the smaller laboratory for these ideas. In bandits, there are no state transitions, so the exploration-exploitation tradeoff is isolated. RL generalizes this problem because information gathered now can affect future states and future information. The discussions in these lectures should be read as asking: what uncertainty matters for decision-making, and how does the algorithm convert uncertainty into action choice?

8. MCTS, ALPHAGO/ALPHAZERO, AND PLANNING WITH LEARNED COMPONENTS

MCTS shows that learning and planning can be combined. Instead of computing a full policy over all states, the agent spends local computation at the current state. Simple Monte Carlo search simulates rollouts for candidate actions. Expectimax is exact lookahead but grows exponentially. MCTS samples selectively, building a tree where promising or uncertain nodes receive more computation. UCT borrows bandit upper-confidence ideas at each tree node.

AlphaGo/AlphaZero-style systems add learned policy priors, learned value estimates, and self-play. Policy networks guide search; value networks evaluate leaves; MCTS improves move selection; self-play produces new training data. The important lesson is not only that Go was solved impressively, but that model-based search, model-free learning, supervised initialization, and self-play curricula can be composed.

9. ETHICS, SOCIETY, AND DEPLOYMENT

The ethics material should be read as part of the technical course, not as an appendix. RL optimizes specified rewards, and specified rewards are proxies. Agents can exploit simulator errors, optimize engagement at social cost, amplify feedback loops, or choose actions that

gather data in harmful ways. Evaluation must include robustness, distribution shift, stakeholder impact, reward design, and monitoring after deployment. ?High return? is not the same thing as ?good system.?

10. WORLD MODELING AND MODEL-BASED RL

The Shane Gu guest lecture on world modeling fits after MCTS and data-efficient RL. Model-based RL tries to learn a predictive model of the world and use it for planning, imagination, or representation learning. The promise is sample efficiency: simulated experience can be cheaper than real experience. The danger is model error: small prediction mistakes can compound under planning, and the agent may exploit model inaccuracies. Good world models need useful abstractions, uncertainty awareness, and careful planning horizons.

CROSS-CUTTING REFERENCES AND PAPERS MENTIONED BY THE COURSE MATERIALS

- Sutton & Barto, *Reinforcement Learning: An Introduction*, 2nd edition: foundational textbook for MDPs, dynamic programming, Monte Carlo methods, TD learning, control, function approximation, and policy gradients.
- CS229 linear algebra and probability reviews: prerequisites for value-function approximation, gradients, uncertainty, and expectation notation.
- Ross et al., DAgger / no-regret reduction: explains why behavior cloning suffers compounding error and why data aggregation helps.
- Bradley-Terry preference modeling: useful for pairwise human preference learning and RLHF pipelines.
- Bandit Algorithms, Section 7.1: formal exploration/regret background for efficient RL.
- Silver et al. AlphaGo / AlphaZero references: canonical examples of self-play, neural policy/value estimates, and tree search integration.
- Applied RL references surfaced in lecture 1: Go, plasma control, COVID-19 border testing, LLM reasoning/RLHF, robotics, tutoring, advertising, and healthcare.

DISCUSSION PROMPTS TO RETAIN

1. For any application, define state, action, reward, transition dynamics, horizon, and observability. If this is hard, the RL problem is underspecified.
2. If planning is possible, ask whether the model is known, learned, or only simulatable.
3. For policy evaluation, decide whether complete episodes are available and whether bootstrapping is acceptable.
4. For control, decide whether the target is on-policy performance or off-policy optimality.
5. For policy gradients, identify the baseline, advantage estimate, and update-size safeguard.
6. For imitation, ask what states the learned policy will visit at test time.
7. For offline RL, identify which actions are unsupported by the dataset.
8. For exploration, specify the uncertainty estimate and how it changes the action choice.
9. For MCTS, identify the simulator/model, rollout/evaluation policy, selection rule, and backup rule.
10. For deployment, ask who is affected by reward optimization and what happens under distribution shift.

COMPACT EXAM MEMORY MAP

- RL: sequential decisions plus feedback from rewards.
- MDP: states, actions, transitions, rewards, discount.
- Return: discounted future reward.

- Value: expected return from a state or state-action pair.
- Bellman equations: recursive definitions of value.
- Planning: compute decisions with a known/simulatable model.
- Policy iteration: evaluate, improve, repeat.
- Value iteration: repeated optimal Bellman backups.
- Monte Carlo: learn from full returns.
- TD: learn from one-step bootstrapped targets.
- SARSA: on-policy TD control.
- Q-learning: off-policy greedy-target control.
- Function approximation: generalization plus instability risk.
- Policy gradient: directly optimize policy parameters.
- Actor-critic: policy actor plus value/advantage critic.
- Imitation: demonstrations; beware covariate shift.
- DAgger: collect expert labels on learner-induced states.
- RLHF/preference learning: learn from human comparisons or feedback.
- Offline RL: fixed data; conservatism/constraints matter.
- Exploration: convert uncertainty into informative actions.
- Bandits: simplest regret laboratory.
- MCTS: simulation-based local planning with tree search.
- World models: learned predictive models for imagination/planning.
- Ethics: reward is a proxy; evaluate consequences.